

Zero-Knowledge Ring-Switching without the Transpose

Andrei Kochergin
Oumuamua Labs
`andrei@oumuamua.dev`

Abstract

We make ring-switching zero-knowledge, closing the problem Diamond left open after making the large-field commitment scheme underlying it zero-knowledge: “We leave for follow-up work the task of making ring-switching zero-knowledge” [3, §5]. Running the verifier of the Diamond and Posen reduction on one-time-padded partial evaluations forces a correction. Over a binary field L of degree n that correction is certified by $n - 1$ multiplication constraints, $2(n - 1)$ affine constraints and $3(n - 1)$ auxiliary variables, for any number m of batched claims, in an outer argument with affine rows and multiplication triples. At $n = 128$ the counts are 127, 254 and 381. We instantiated the coordinate-matrix gadget of the Binius64 zero-knowledge wrapper at the same n , which commits the $n \times n$ coordinate matrix and constrains it by subfield membership, and measured 16 384 multiplications between hidden wires. The reduction never forms the transpose its second check appears to require: it consumes only the row-batched image of that transpose, which is the value of a single linearized polynomial; moving the Frobenius from the hidden entries onto the public weights merges the m claims into n affine forms, and Horner’s rule in the Frobenius evaluates the family in $n - 1$ squarings. The linearized form of coordinate extraction is Soukhanov’s and the padded reduction instantiates a known compiler; new here are the instantiation over hidden operands, its completeness and uniqueness proof, and the measurement. The constraints carry about n nonzero public coefficients per claim, and we treat packing over $\mathbb{F}_2$ only.

1 Introduction

The transpose that ring-switching’s second check appears to require need never be formed. The reduction uses only its contraction against a fresh challenge, and that contraction is the value of a single linearized polynomial at each hidden entry.

Ring-switching [5, Construction 3.1] reduces an evaluation claim on a multilinear over a tiny binary field to a claim on its packed image over a large extension L , with no embedding overhead, and it lets any large-field commitment scheme commit bit-valued witnesses over towers of binary fields [4] in packed form. Its verifier receives a vector of partial evaluations, one element of L per packed coordinate, and applies two checks to it. The first is L -linear. The second is not: it reads the same data transposed, and transposition means extracting the $\mathbb{F}_2$ -coordinates of each entry.

In the clear this costs nothing. Diamond made the large-field scheme zero-knowledge [3] and left the reduction open: “We leave for follow-up work the task of making ring-switching zero-knowledge” [3, §5]. BinarySpartan, which instantiates Spartan over $\mathbb{F}_2$ with ring-switching, records the same gap from the other side: “Binius64 and both Vega configurations are zero-knowledge, which BinarySpartan and Flock are not” [8, §3.2]. It applies its bit-valued linear maps by “position-specific byte lookup tables” [8, §1] on cleartext values.

Under a one-time pad the second check is the difficulty. The verifier holds each partial evaluation $\hat{s}_v$ only as $\hat{s}_v + p_v$ with the pad entry p_v hidden, coordinate extraction is $\mathbb{F}_2$ -linear, and the correction it must account for is a function of the pad alone. That correction has to be certified against a commitment to the pad, inside an outer argument whose primitives are affine rows with public coefficients and multiplication triples. Coordinate extraction is not an affine row, and the usual remedy is to commit the coordinates themselves, n hidden variables and n subfield-membership constraints per claim, with n the degree of L over $\mathbb{F}_2$. This is what the Binius64 zero-knowledge wrapper does.

Linearized polynomials are built from squarings and multiplications by constants, which are exactly the primitives the outer argument has. Moving the Frobenius from the hidden entries onto the public weights makes the number of resulting affine forms independent of how many claims are batched, and Horner’s rule in the Frobenius evaluates the whole family in $n - 1$ squarings. Theorem 5 gives the constraint system, $n - 1$ multiplication constraints, $2(n - 1)$ affine constraints and $3(n - 1)$ auxiliary variables for a binary field of degree n and any number of batched claims, and proves that for every value of the hidden entries it has exactly one satisfying assignment, which yields the correction. Theorem 8 builds the padded reduction around it and proves that reduction honest-verifier zero-knowledge whenever the outer argument is, a hypothesis [1] discharges. At $n = 128$ that is 127 multiplications and 381 variables. The coordinate-matrix gadget of the Binius64 wrapper, instantiated through its public interface at the same n in Section 8, spends 48 896 multiplication constraints and 48 768 private variables on one tensor element, 16 384 of those constraints being between two hidden wires.

The algebra is classical and both devices are in Soukhanov’s Hashcaster note. That coordinate extraction against a trace-dual basis is a linearized polynomial is in [7, §3.4], and [9] states it in that form under the heading “Linearized formula for π ”; the same note moves the Frobenius onto public weights to batch a Frobenius orbit of opening points, and the Binius64 specification performs the same transfer [10, §4.5.4]. Evaluation by nested squaring is Horner’s rule in the twisted polynomial ring.

Closest is [5, §1.4], which reproduces Soukhanov’s technique in the ring-switching setting itself. It writes the matrix whose (u, v) entry is $\sigma^u(b_v)$, applies it to the claimed vector, cites [7, Lem. 3.51] for its nonsingularity, and relates the result to the transposition of Section 4. There the verifier forms that $n \times n$ matrix and evaluates it on public data. What differs here is contracting against the batching weights before extracting coordinates, which replaces the matrix by one linearized polynomial with n coefficients, and evaluating that polynomial on hidden operands inside a constraint system.

Section 7 is likewise an instance of a known compiler: pad the values a verifier consumes, publish them masked, and certify that verifier’s predicate on the plaintexts in an outer argument. Frigo and Shelat state it for Ligerio [6], and [2] and [10, §7.1] do the same. The instantiation over hidden operands, its completeness and uniqueness proof, and the measurement against the shipped alternative are what this note adds.

We treat the packing field $\mathbb{F}_2$ only. For a larger packing field the functional is the relative trace and the chain is shorter, a separate construction that we do not give. The independence from m concerns the number of constraints and variables: each of the n affine forms carries one public coefficient per claim, and both parties evaluate them.

2 Finite fields

Throughout, $\mathbb{F} = \mathbb{F}_2$ and $L = \mathbb{F}_{2^n}$, a binary field of degree n over $\mathbb{F}$. Write $\sigma: L \rightarrow L$ for the Frobenius automorphism $\sigma(x) = x^2$. It generates $\text{Gal}(L/\mathbb{F})$ and satisfies $\sigma^n = \text{id}$,

whence $\sigma^{-j} = \sigma^{n-j}$.

Define the trace $\text{Tr}: L \rightarrow \mathbb{F}$ by

$$\text{Tr}(x) = \sum_{j=0}^{n-1} \sigma^j(x).$$

It is $\mathbb{F}$ -linear, surjective onto $\mathbb{F}$, and the pairing $\langle x, y \rangle = \text{Tr}(xy)$ is nondegenerate [7, §2.3]. Every $\mathbb{F}$ -basis of L therefore has a dual. Fix a basis $b_0, \dots, b_{n-1}$ and let $\gamma_0, \dots, \gamma_{n-1}$ be its trace-dual, meaning $\text{Tr}(\gamma_u b_w) = \delta_{uw}$. Write $x = \sum_u c_u(x) b_u$ with $c_u(x) \in \mathbb{F}$ for the coordinates of x .

Lemma 1. $c_u(x) = \text{Tr}(\gamma_u x)$ for every u and every $x \in L$.

Proof. Expand $x = \sum_w c_w(x) b_w$. Since Tr is $\mathbb{F}$ -linear and $c_w(x) \in \mathbb{F}$,

$$\text{Tr}(\gamma_u x) = \sum_w c_w(x) \text{Tr}(\gamma_u b_w) = c_u(x). \quad \square$$

A polynomial over L of the form $\sum_j \mu_j X^{2^j}$ is linearized [7, §3.4], and it induces an $\mathbb{F}$ -linear map $L \rightarrow L$. Distinct linearized polynomials of degree below 2^n induce distinct maps, since their difference is a nonzero polynomial with fewer than 2^n roots, and as both spaces have dimension n^2 over $\mathbb{F}$, every $\mathbb{F}$ -linear map $L \rightarrow L$ arises in this way. Two consequences are used below. The trace is itself linearized, with every coefficient equal to 1. The L -linear maps $L \rightarrow L$ are exactly those of the form $\mu_0 X$, a one-dimensional subspace of the n -dimensional L -space of all $\mathbb{F}$ -linear maps.

3 The batched image

We recall only what is needed of [5, Construction 3.1]. The prover sends the vector of partial evaluations $\hat{s} = (\hat{s}_v)_v$, one element of L per packed coordinate. The verifier checks it against the incoming claim by an L -linear relation, draws a challenge r'' , and forms

$$s_0 = \sum_u \text{eq}(u, r'') \cdot \hat{s}_u, \quad (1)$$

where $(\hat{s}_u)_u$ is the row representation of the tensor-algebra element whose column representation is $(\hat{s}_v)_v$. The scalar s_0 is the initial claim of the sumcheck that closes the reduction, and it is the only quantity the reduction consumes; the individual $\hat{s}_u$ are not used again. This is visible in the reference implementation, which forms the transpose and, after sampling r'' , contracts it against r'' [11].

Passing between the two representations is coordinate extraction applied entrywise. Writing $e_u = \text{eq}(u, r'')$ and applying Lemma 1, equation (1) becomes

$$s_0 = \sum_v b_v \cdot \varphi(\hat{s}_v), \quad \varphi(x) = \sum_u e_u c_u(x). \quad (2)$$

4 The obstruction

The transposed data cannot be dispensed with, and outside an exceptional case the map that extracts it is not an affine row.

The single element $\sum_v \hat{s}_v b_v$ does not determine the reduction's input. Diamond and Posen record this as [5, Remark 3.3], with an arbitrary subfield of L in the role of $\mathbb{F}$: it is

the image of $\hat{s}$ under the canonical multiplication map $L \otimes_{\mathbb{F}} L \rightarrow L$, which “is of course not injective”, and reflects $\hat{s}$ “only lossfully”. The transposed data is required.

The map φ of (2) is $\mathbb{F}$ -linear, and it is L -linear only exceptionally. By the previous section the L -linear maps $L \rightarrow L$ are the $\mu_0 X$, one dimension inside the n the $\mathbb{F}$ -linear ones occupy, and Lemma 3 below names the coefficients of φ : it is L -linear exactly when $\mu_j = 0$ for every $j \geq 1$, which is $n - 1$ conditions on the challenge. Nothing below decides that question, the construction being correct either way, and it degenerates gracefully in the exceptional case.

Suppose now the prover transmits $\hat{s}'_v = \hat{s}_v + p_v$ with p_v hidden. The verifier can evaluate $\varphi(\hat{s}'_v)$ but needs $\varphi(\hat{s}_v)$, and by $\mathbb{F}$ -linearity these differ by $\varphi(p_v)$, which depends on the pad alone. A verifier that has only a commitment to the pad must therefore certify

$$\Delta = \sum_{c=1}^m a_c \varphi(h_c) \quad (3)$$

inside the outer argument, where $h_1, \dots, h_m \in L$ are the hidden pad entries, one per hidden claim, and $a_1, \dots, a_m \in L$ are public. Whenever some μ_j with $j \geq 1$ is nonzero, no affine row expresses φ .

Remark 2. The Binius64 specification states at [10, §4.8, step 4] that the reduction’s target is “a fixed K -linear combination of the entries of $\hat{s}$ ”, where K is that document’s large field, our L . That is not what the implementation computes, and by the count above it cannot be. A tensor-algebra element carries two L -structures, and the target is indeed an L -linear combination of the entries of the row representation; the reading is fixed by step 1 of the same subsection, which sends $\hat{s}$ as the vector of partial evaluations indexed by the packed coordinates. We cite the implementation with a commit hash throughout.

5 The batching identity

The correction Δ of (3) decomposes into n affine forms in the hidden entries, one per power of the Frobenius, with public coefficients and no dependence on m .

Lemma 3. $\varphi(x) = \sum_{j=0}^{n-1} \mu_j \sigma^j(x)$, where $\mu_j = \sum_u e_u \sigma^j(\gamma_u)$.

Proof. By Lemma 1, $\varphi(x) = \sum_u e_u \text{Tr}(\gamma_u x) = \sum_u e_u \sum_j \sigma^j(\gamma_u x)$. As σ is a ring homomorphism, $\sigma^j(\gamma_u x) = \sigma^j(\gamma_u) \sigma^j(x)$. Exchange the order of summation. $\square$

Lemma 4. Put $\tau_{j,c} = \sigma^{-j}(\mu_j a_c)$ and $T_j = \sum_c \tau_{j,c} h_c$. Then $\Delta = \sum_{j=0}^{n-1} \sigma^j(T_j)$.

Proof. By Lemma 3, $\Delta = \sum_c a_c \sum_j \mu_j \sigma^j(h_c) = \sum_j \sum_c \mu_j a_c \sigma^j(h_c)$. Since σ is an automorphism and $\sigma^j \sigma^{-j} = \text{id}$,

$$\sum_c \mu_j a_c \sigma^j(h_c) = \sum_c \sigma^j(\sigma^{-j}(\mu_j a_c)) \sigma^j(h_c) = \sigma^j\left(\sum_c \sigma^{-j}(\mu_j a_c) h_c\right) = \sigma^j(T_j). \quad \square$$

Each $\tau_{j,c}$ is public, hence each T_j is an affine form in the hidden h_c with public coefficients. There are n such forms whatever m may be. This is the step that removes the dependence on the number of claims, and it is the device of [9, “Improvement 1”] and [10, §4.5.4] applied on a different index.

Unrolling $\sum_j \sigma^j(T_j)$ from the top gives $\Delta = T_0 + \sigma(T_1 + \sigma(T_2 + \dots))$, which is Horner’s rule with σ in place of multiplication by the indeterminate. Define

$$X_{n-1} = T_{n-1}, \quad X_j = T_j + \sigma(X_{j+1}) \quad (0 \leq j \leq n-2). \quad (4)$$

Then $X_0 = \Delta$.

6 The constraint system

Suppose the outer argument exposes hidden variables and enforces two kinds of constraint, affine ones

$$\alpha + \sum_i \alpha_i z_i = 0$$

with α and the α_i public in L , and multiplication constraints $u \cdot v = w$ on hidden triples (u, v, w) . This is the interface of [1].

Theorem 5. *Let $n \geq 2$, let $h_1, \dots, h_m$ be hidden variables, and let μ_j and a_c be public. There is a system $\mathcal{C}$ over $3(n-1)$ further hidden variables, consisting of $2(n-1)$ affine constraints and $n-1$ multiplication constraints, and an affine form D with public coefficients over the h_c and the further variables, such that for every assignment of the h_c the assignment of the further variables satisfying $\mathcal{C}$ exists, is unique, and gives $D = \Delta$ with Δ as in (3). All three counts are independent of m .*

Proof. For $j = 1, \dots, n-1$ introduce a triple (ℓ_j, r_j, q_j) , giving $3(n-1)$ variables, and impose

$$\ell_j + r_j = 0, \tag{5}$$

$$\ell_j + [j \leq n-2] q_{j+1} + \sum_c \tau_{j,c} h_c = 0, \tag{6}$$

$$\ell_j \cdot r_j = q_j, \tag{7}$$

where $[\cdot]$ is 1 when the condition holds and 0 otherwise. All arithmetic is in characteristic two and signs are immaterial. Constraints (5) and (6) are affine and number $2(n-1)$; constraint (7) is a multiplication constraint and there are $n-1$ of them. Put $D = q_1 + \sum_c \tau_{0,c} h_c$. There is no triple at $j = 0$, since X_0 is never squared: it enters only as D , which is (4) at $j = 0$ with q_1 in place of $\sigma(X_1)$. No constraint has degree above two, hence the outer argument's degree is unchanged.

For existence, take $\ell_j = r_j = X_j$ with X_j as in (4) and $q_j = X_j^2 = \sigma(X_j)$. Then (5) and (7) hold by construction. For $j \leq n-2$, constraint (6) reads $X_j + \sigma(X_{j+1}) + T_j = 0$, which is (4); for $j = n-1$ it reads $X_{n-1} + T_{n-1} = 0$, likewise. Then $D = \sigma(X_1) + T_0 = X_0 = \Delta$.

For uniqueness, fix the h_c and hence the T_j , and induct downward on j . At $j = n-1$ the indicator in (6) vanishes, since no variable q_n exists, and the constraint reads $\ell_{n-1} = T_{n-1}$. This is the base case, and it is the step on which the whole chain rests. Suppose ℓ_{j+1} is determined. By (5), $r_{j+1} = \ell_{j+1}$, and by (7), $q_{j+1} = \ell_{j+1}^2 = \sigma(\ell_{j+1})$; both are then determined. Constraint (6) at index j now reads $\ell_j = T_j + q_{j+1}$, which determines ℓ_j , and (5) and (7) determine r_j and q_j . Since the satisfying assignment is unique and the one exhibited above satisfies $\mathcal{C}$, every satisfying assignment gives $D = \Delta$.

Characteristic two enters through (7): squaring is the Frobenius σ , hence additive, and one multiplication constraint computes it. The induction never inverts a square. $\square$

Theorem 5 concerns the constraint system alone. Composed with the outer argument it inherits that argument's soundness error; the equality $D = \Delta$ holds unconditionally for satisfying assignments, and the outer argument's error governs the probability that a non-satisfying assignment is accepted.

Remark 6. The m claims need not share a functional. Let claim c carry its own φ_c , with coefficients $\mu_{j,c}$ by Lemma 3, and put $\tau_{j,c} = \sigma^{-j}(\mu_{j,c} a_c)$. Lemma 4 and Theorem 5 then read unchanged, all three counts with them: one chain absorbs claims drawn against unrelated challenges. This is what lets m exceed n . A single reduction contributes exactly $[L : \mathbb{F}]$

claims, one per packed coordinate, and a system batching many reductions into one outer argument contributes that many times over.

7 The padded reduction

The construction below pads the values the verifier consumes and certifies its predicate on the plaintexts, which is the compiler of [6].

Write λ for the L -linear form in which the verifier's first check reads $\lambda(\hat{s}) = y$, with y the incoming claim, and index the packed coordinates by $v = 1, \dots, m$. Both ends of the reduction are masked: y arrives as $y' = y + p_{\text{in}}$, and the hand-off claim s_0 of (1) leaves as $s'_0 = s_0 + p_0$. The pad entries p_{in} , p_0 and $p = (p_1, \dots, p_m)$ are hidden variables of the outer argument of Section 6, and p_0 occurs in no other of its rows.

1. The prover publishes $\hat{s}' = (\hat{s}_v + p_v)_v$.
2. The verifier draws r'' .
3. The prover publishes s'_0 . Both parties form μ_j from r'' and $\tau_{j,v} = \sigma^{-j}(\mu_j b_v)$, and the verifier forms $\Sigma = \sum_v b_v \varphi(\hat{s}'_v)$.
4. They run the outer argument on the system $\mathcal{C}$ of Theorem 5, taken at $h_v = p_v$ and $a_v = b_v$, together with the two affine rows

$$\lambda(p) + p_{\text{in}} = \lambda(\hat{s}') + y', \quad (8)$$

$$D + p_0 = \Sigma + s'_0. \quad (9)$$

Both right-hand sides are public. No transpose is formed at any point. Both parties already derive the μ_j in step 3, and the verifier evaluates φ on the published entries through Lemma 3 rather than by extracting their coordinates; on the hidden entries the correction that extraction would have yielded is the affine form D , at the cost of Theorem 5.

The honest assignment satisfies both rows. Row (8) is L -linearity of λ applied to $\hat{s}' = \hat{s} + p$. In row (9) the $\mathbb{F}$ -linearity of φ gives $\Sigma = s_0 + \Delta$ by (2) and (3), and $D = \Delta$ by Theorem 5.

Lemma 7. *If $p_0, p_1, \dots, p_m$ are uniform on L and independent, then $(\hat{s}', s'_0)$ is uniform on L^{m+1} and independent of $\hat{s}$.*

Proof. Each of $p_1, \dots, p_m, p_0$ masks one published value and no other. A uniform summand independent of the addend gives a uniform sum independent of it, and the $m + 1$ sums are independent because their summands are. $\square$

Theorem 8. *Let the pad be as in Lemma 7. If the outer argument of Section 6 is honest-verifier zero-knowledge, then the reduction above is honest-verifier zero-knowledge: there is a simulator that reads the public data and the verifier's coins but not $\hat{s}$, and whose output lies at the outer argument's own simulation distance from the real view. The claim is about the reduction alone, the large-field commitment it opens and the sumcheck receiving s_0 being separate components, as in [3]. With code dimension exceeding message length plus the number of queried columns, the interactive PCP of [1] is perfect honest-verifier zero-knowledge [1, Lemma 4.15], which discharges the hypothesis and sets that distance to zero. Any other outer argument owes its own proof of the hypothesis, matching that geometry being necessary and not sufficient.*

Proof. Write ρ for the outer argument's coins and S_{out} for its simulator. The simulator S draws $\tilde{s}'$ uniform on L^m and $\tilde{s}'_0$ uniform on L , forms the $\tau_{j,v}$ and Σ from r'' and $\tilde{s}'$, assembles $\mathcal{C}$ and the rows (8) and (9) with $\tilde{s}'$ and $\tilde{s}'_0$ in place of the published values, runs S_{out} on that system at coins ρ , and outputs $\tilde{s}'$, r'' , $\tilde{s}'_0$ and the transcript returned. The system is fixed only once r'' is known, and a simulator holding the coins in advance holds it as well. Nothing S reads depends on $\hat{s}$.

The assembled system is satisfiable for every pair drawn. As $\lambda \neq 0$ is onto L , some $p \in L^m$ has $\lambda(p) = \lambda(\tilde{s}') + y' + p_{\text{in}}$, which is (8); Theorem 5 determines the auxiliary variables and hence D from that p ; and $p_0 := D + \Sigma + \tilde{s}'_0$ satisfies (9), being unconstrained elsewhere. The guarantee on S_{out} therefore applies.

In the real view r'' is the verifier's own coin, and $(\tilde{s}', s'_0)$ is uniform on L^{m+1} and independent of it and of $\hat{s}$ by Lemma 7. Conditioned on those three, the rest of the view is the outer argument's transcript at coins ρ on the system they determine. S draws the three from that law and the fourth from S_{out} on the same system, whence the two views differ by the outer argument's simulation distance. $\square$

Remark 9. The reduction is public-coin, and the standard random-oracle argument carries Theorem 8 to its Fiat-Shamir compilation: the simulator fixes its published values first, then programs the oracle at each queried prefix. Deriving the pad from a seeded generator rather than from uniform coins replaces the equality of Lemma 7 by that generator's distinguishing advantage.

8 Costs

The alternative is to commit the coordinates. For each hidden claim one introduces n variables, forces each into the subfield, and ties them to the claim by an affine constraint; φ is then affine in those variables. Over $\mathbb{F}_2$ the subfield constraint is $z^2 = z$. Per tensor element this is n^2 multiplication constraints and n^2 variables. A constraint system whose linear combinations carry no scalar coefficients pays a further $2n(n-1)$ multiplication constraints and $2n(n-1) - n$ variables for the scalings by basis constants.

This is what the Binius64 zero-knowledge wrapper does. Its verifier transposes the tensor-algebra element explicitly, at `crates/verifier/src/ring_switch.rs:69` in commit 5eafd191 [11], and that verifier is executed symbolically against a constraint builder to produce the outer system, at `crates/verifier/src/zk_config.rs:83`. The transposition resolves to a gadget which commits the $n \times n$ coordinate matrix, checks membership by k -fold Frobenius with k the ratio of the field degree to n , checks reconstruction, and emits the transpose, at `crates/spartan-verifier/src/wrapper/gadgets.rs:29`.

We instantiated that gadget on $n = 128$ hidden inputs through its public interface, without modifying its repository, compiled it along the path `ZKVerifier::setup` takes (`compile`), and read off the resulting system. It has 48 896 multiplication constraints and 48 768 private variables, that is $3n^2 - 2n$ and $3n^2 - 3n$: 16 384 membership squarings between hidden wires and 32 512 scalings of a hidden wire by a public basis constant. Theorem 5 at $n = 128$ gives 127 and 381. In the model of Section 6, whose affine rows carry public coefficients, only the 16 384 are multiplication constraints, and the 127 stands against those. The difference in scope matters more than the ratio: the coordinate-matrix gadget is instantiated once per tensor-algebra element, which a system hiding one packed claim per proof pays once, while a system hiding one claim per packed column per row offset pays it per claim.

Table 1 collects the counts.

	coordinate matrix [11]	Theorem 5
multiplication constraints	48 896	127
between two hidden wires	16 384	127
hidden auxiliary variables	48 768	381
instances required	one per tensor element	one for all m claims*

Table 1: Counts at $n = 128$. The left column is measured at commit `5eafd191` after wire elimination; the right is $n - 1$ and $3(n - 1)$. *In constraints and variables only; the nonzero public coefficients number $nm + 4n - 4$.

The independence from m in Theorem 5 concerns the number of constraints and variables. It does not concern the number of nonzero public coefficients in those constraints. Each of the n forms T_j carries one coefficient $\tau_{j,c}$ per hidden claim, and the chain carries $4n - 4$ further coefficients on its own wires, hence $\mathcal{C}$ has $nm + 4n - 4$ of them, and both parties derive and consume them, since both build the same system from public data. Per claim that is $n + (4n - 4)/m$, which at $n = 128$ falls from 132.0 at $m = n$, a single reduction, to 128.03 at $m = 18\,432$, where the total is 2 359 804 and the batch is 144 reductions.

9 Remarks

Nothing in the argument depends on the choice of $\mathbb{F}$ -basis or on $n = 128$: for a binary field of any degree n and any $\mathbb{F}$ -basis, γ is that basis’s trace-dual and the chain has $n - 1$ squarings. An implementation will fix n and may compute γ in closed form from a particular tower basis; these are properties of the instantiation and not of Theorem 5.

The construction uses throughout that the coordinates $c_u(x)$ lie in the prime field, which makes the absolute trace the right functional and fixes the chain length at n . For a packing field $\mathbb{F}_{2^d}$ with $d > 1$ the corresponding object is the relative trace $\text{Tr}_{L/\mathbb{F}_{2^d}}$, the chain has n/d terms in steps of σ^d , and the coefficients differ. We have not written that case out.

The composed argument inherits the outer argument’s soundness error, and analysing that error for a particular outer argument lies outside the scope of this note. The outer argument’s hiding enters as the hypothesis of Theorem 8.

Acknowledgements

The linearized formulation recalled here is Lev Soukhanov’s, and we thank him for making the Hashcaster note public.

References

- [1] Scott Ames, Carmit Hazay, Yuval Ishai, and Muthuramakrishnan Venkatasubramanian. Liger: lightweight sublinear arguments without a trusted setup. *Designs, Codes and Cryptography*, 2023.
- [2] Rahul Dalal, Tamir Hemo, Eugene Rabinovich, and Ron D. Rothblum. VEIL: Lightweight zero-knowledge for hash-based multilinear proof systems. *Cryptology ePrint Archive*, Paper 2026/683, 2026.
- [3] Benjamin E. Diamond. Zero-knowledge polynomial commitment in binary fields. *Cryptology ePrint Archive*, Paper 2025/1015, 2025.

- [4] Benjamin E. Diamond and Jim Posen. Succinct arguments over towers of binary fields. Cryptology ePrint Archive, Paper 2023/1784, 2023. EUROCRYPT 2025.
- [5] Benjamin E. Diamond and Jim Posen. Polylogarithmic proofs for multilinear over binary towers. Cryptology ePrint Archive, Paper 2024/504, 2024.
- [6] Matteo Frigo and abhi shelat. Anonymous credentials from ECDSA. IACR Communications in Cryptology. Implementation at <https://github.com/google/longfellow-zk>, 2024.
- [7] Rudolf Lidl and Harald Niederreiter. *Finite Fields*, volume 20 of *Encyclopedia of Mathematics and Its Applications*. Cambridge University Press, second edition, 1997.
- [8] Srinath Setty. BinarySpartan: Spartan over binary fields. Cryptology ePrint Archive, Paper 2026/1656, 2026.
- [9] Lev Soukhanov. Hashcaster: Frobenius sumcheck for vectorized boolean ops. HackMD note, <https://hackmd.io/@levs57/SJ4fuZMD0>, 2024. Accessed August 17, 2026. The note carries no title; the title given is that of the accompanying repository, <https://github.com/morgana-proofs/hashcaster>.
- [10] The Binius Developers. Binius64 Blueprint. Specification, version of August 12, 2026, <https://www.binius.xyz/blueprint/>, 2026.
- [11] The Binius Developers. Binius64 reference implementation. Source repository <https://github.com/binius-zk/binius64>, commit 5eafd191, 2026.